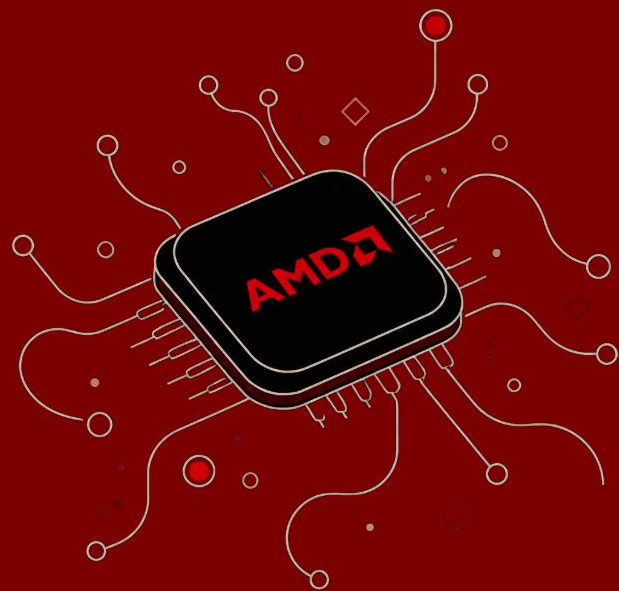


AMD AI DEVMASTER HACKATHON 2026 — TRACK 2



ToolGuard

Real-Time Detection & Repair of Broken Tool Calls on Local AMD Hardware

Team JINX · Local AI Agent Reliability Layer

THE PROBLEM

Silent Agent Failures Are a Production Crisis

AI agents generate malformed, truncated, or schema-leaking tool calls — and most frameworks raise **no error**. The call simply vanishes.

continuedev/continue#5508

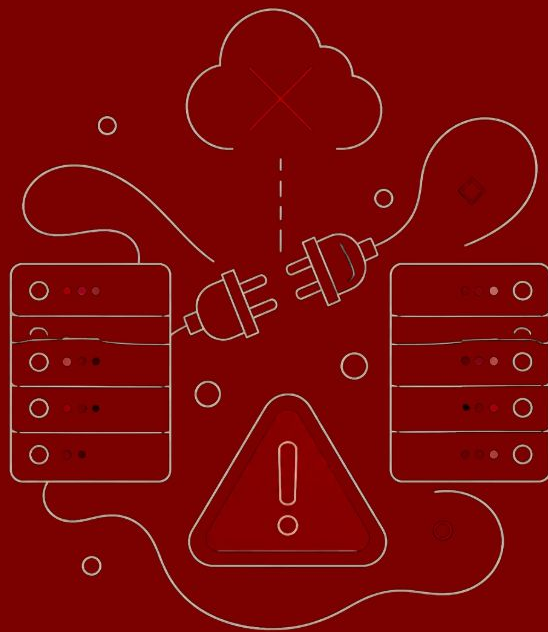
Valid tool calls silently ignored — zero errors raised

GLM-5.1 Discussion #1

Schema definitions leaked; incorrect tool names generated

ROCm#4909 & #6148

Device errors and multi-GPU failures — still open



THE SOLUTION

ToolGuard: The AI Switchboard Operator

ToolGuard sits between the agent and local tool execution — watching every call, catching failures, and repairing them in real time without manual debugging.



Watches

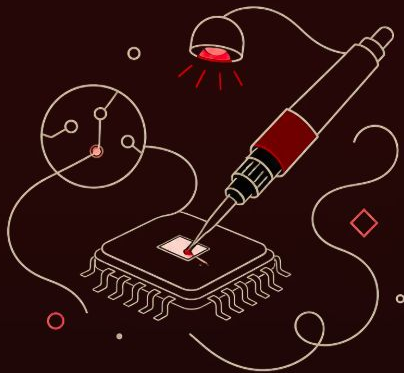
Detects

Repairs

Displays

Every broken call is recovered automatically — no blind retries, no silent drops.

Targeted Repair, Not a Blind Retry



How Repair Works

- Classifies the exact failure pattern
- Issues specific corrective feedback to the model
- Verifies output validity before execution

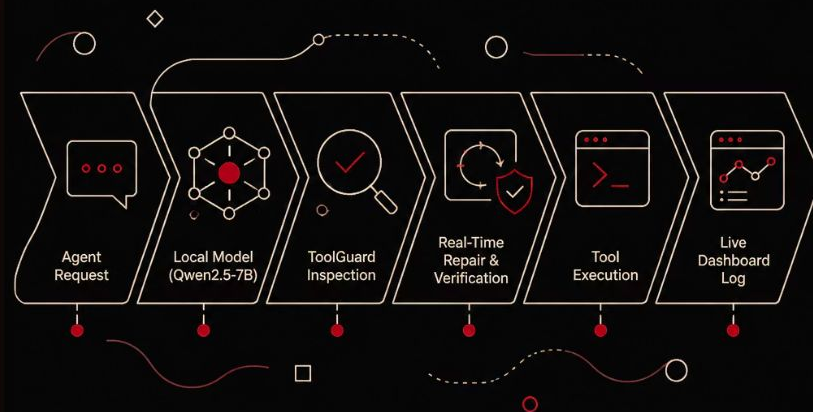
Supported Capabilities

- **Tool Invocation:** Executes local simulated tools — search, calculate, get_weather, read_file
- **Permission Control:** detector.py enforces a real-time allow-list, rejecting unapproved tool names

End-to-End Flow

The agent sends a request to the local model. If the response is a valid tool call, ToolGuard passes it through untouched. If broken, ToolGuard detects the failure, hands it back to the model with corrective instructions, verifies the repair, and logs every outcome to the live dashboard.

- ❏ ToolGuard adds **zero latency** to valid calls — inspection is instantaneous. Repair only activates on failure.



BENCHMARK RESULTS

Reliability: 32% → 100%

50 real trials on live AMD hardware running Qwen2.5-7B-Instruct — every trial is a real API call, nothing scripted or faked.

32%

Without ToolGuard

Baseline success rate across all failure patterns

100%

With ToolGuard

Every trial successfully completed

+68

Reliability Gain

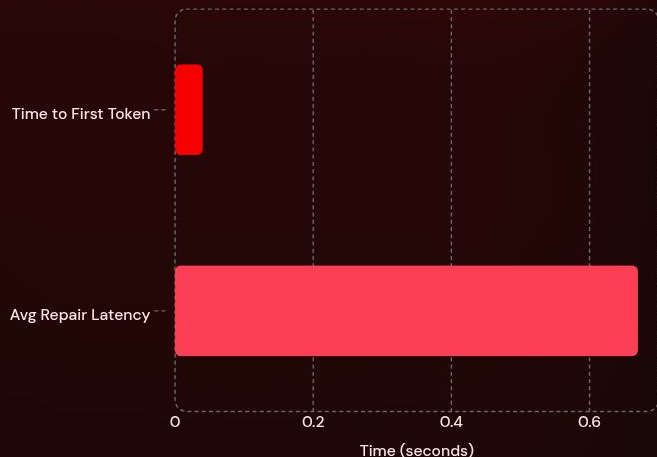
Percentage points of absolute improvement

Evaluated against 3 core reliable patterns (plain text output, schema leaks, dropped streams) plus 1 bonus edge case (malformed tool names).

PERFORMANCE

Speed & Latency: Production-Ready

Metric



Inference Speed

Steady-state, measured over 5 real trials:

- **TTFT:** ~0.04 seconds
- **Generation:** ~32.8 tokens/sec

Repair Overhead

Average end-to-end repair latency across all 3 core failure patterns (9 trials, all repaired):

- **~0.67 seconds** — classify, correct, and verify

ROCm Optimizations (confirmed in server logs)

- Chunked prefill (max_num_batched_tokens=2048)
- Prefix caching enabled
- Ahead-of-time CUDA graph compilation via vLLM



AMD & ROCM

Native AMD Radeon & ROCm Optimization

Engine

Qwen2.5-7B-Instruct via vLLM on ROCm Triton Attention backend — confirmed in server logs, not a fallback

Ecosystem Fix

Solved the `pip install vllm` CUDA dependency trap via automated `bootstrap.sh` environment bootstrapping

VRAM Footprint

~41.3 GiB / 48 GiB (14.37 GiB weights + ~27 GiB KV cache); ToolGuard codebase: 128 KB, zero extra memory

Persistent Caching

Model reload time reduced from ~3 minutes to seconds across instance restarts

HONEST ASSESSMENT

Limitations & Future Roadmap

Current Scope Boundaries

- **Dashboard UI:** Terminal-based due to cloud platform port-forwarding constraints — fully functional for all testing and demos
- **Layer Scope:** Operates at the application/developer layer — driver/kernel-level integration is a long-term vision, not a hackathon scope

Next Steps

- Benchmark smaller, quantized local models for low-resource devices
- Web-based dashboard once port-exposure constraints are resolved
- Expand tool allow-list and permission control policies
- Broader model compatibility testing beyond Qwen2.5-7B

TEAM JINX

Try It Yourself

Step 1 – Bootstrap

```
bash scripts/bootstrap.sh
```

Installs system packages and Python dependencies. Safe to re-run.

Step 3 – Benchmark

```
python3 benchmarks/run_benchmark.py
```

Reproduce the 32% → 100% reliability result on your own hardware.

Step 2 – Launch

```
scripts/start_toolguard.sh
```

Starts the model, live traffic loop, and terminal dashboard — one command.

Team JINX: Joshua M Abraham · Jerin Saji · Jino J. — AMD AI DevMaster Hackathon 2026, Track 2: Agentic AI

